

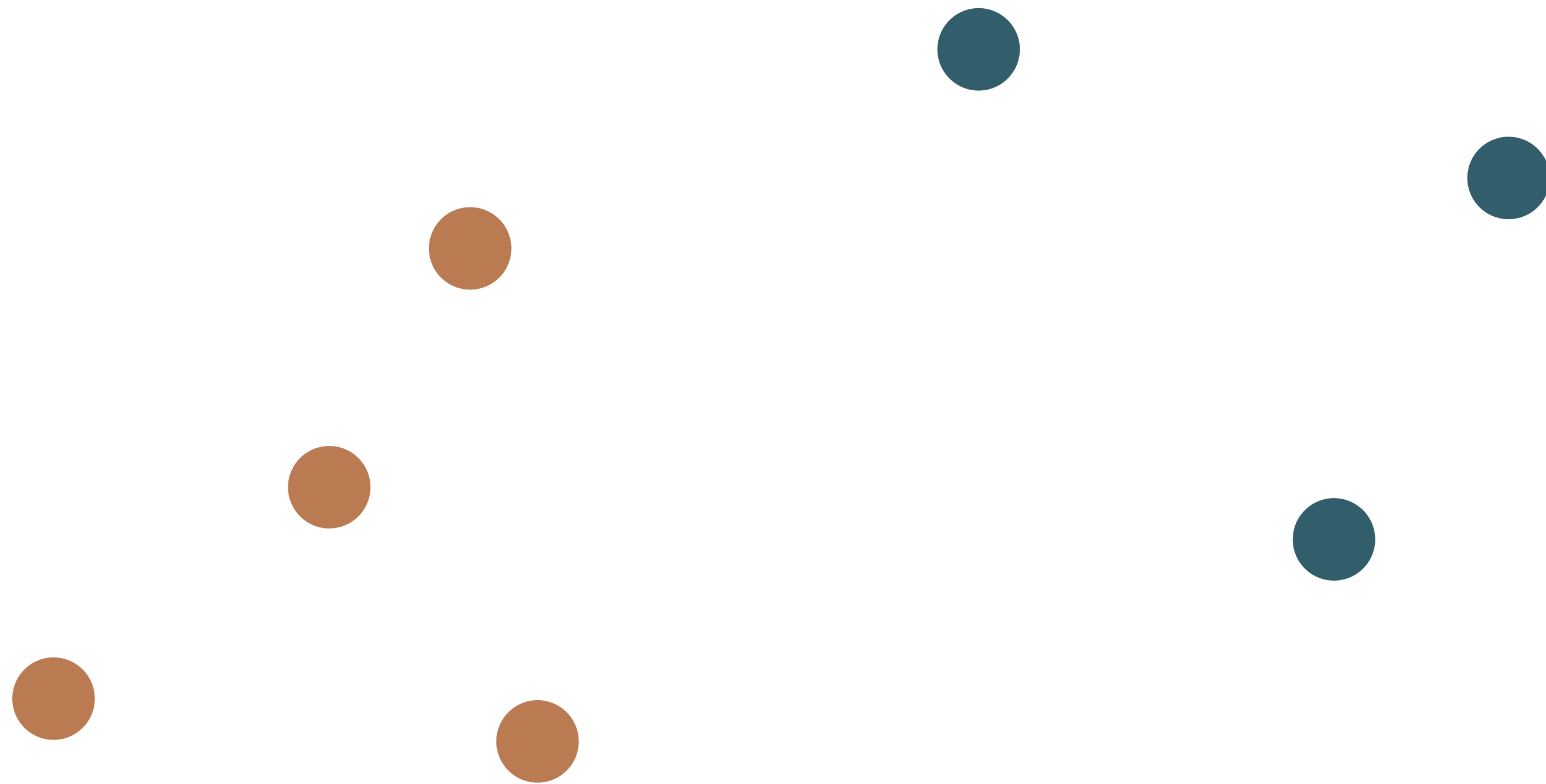
Advanced Neural Network Training Techniques

CSC 487: Deep Learning
Instructor: Jonathan Ventura

Data augmentation

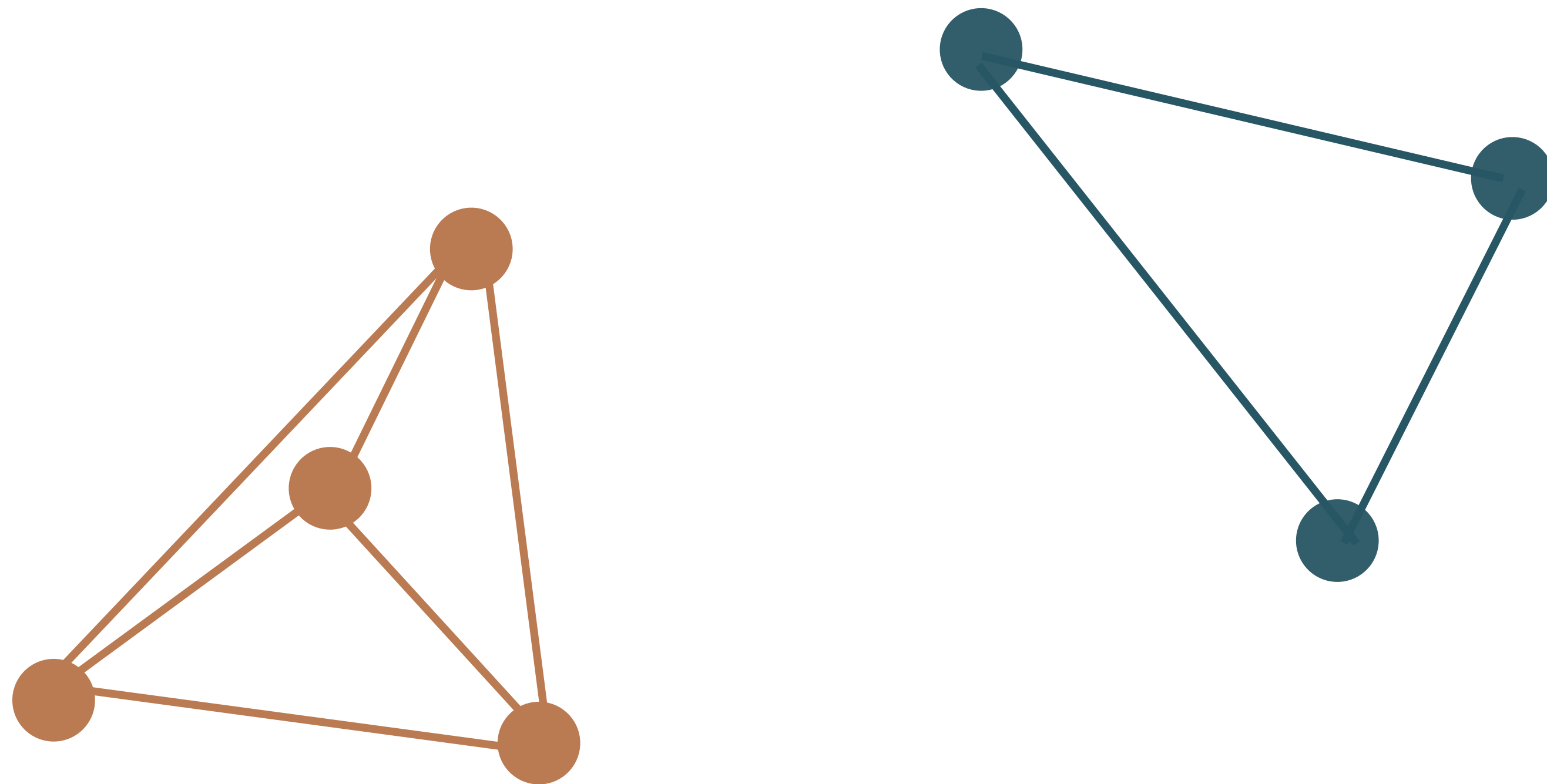
- Best way to combat overfitting is to add training data. Why?
- But training data is expensive to obtain.
- Idea: create new examples from existing training data.

SMOTE: Synthetic Minority Over-sampling Technique



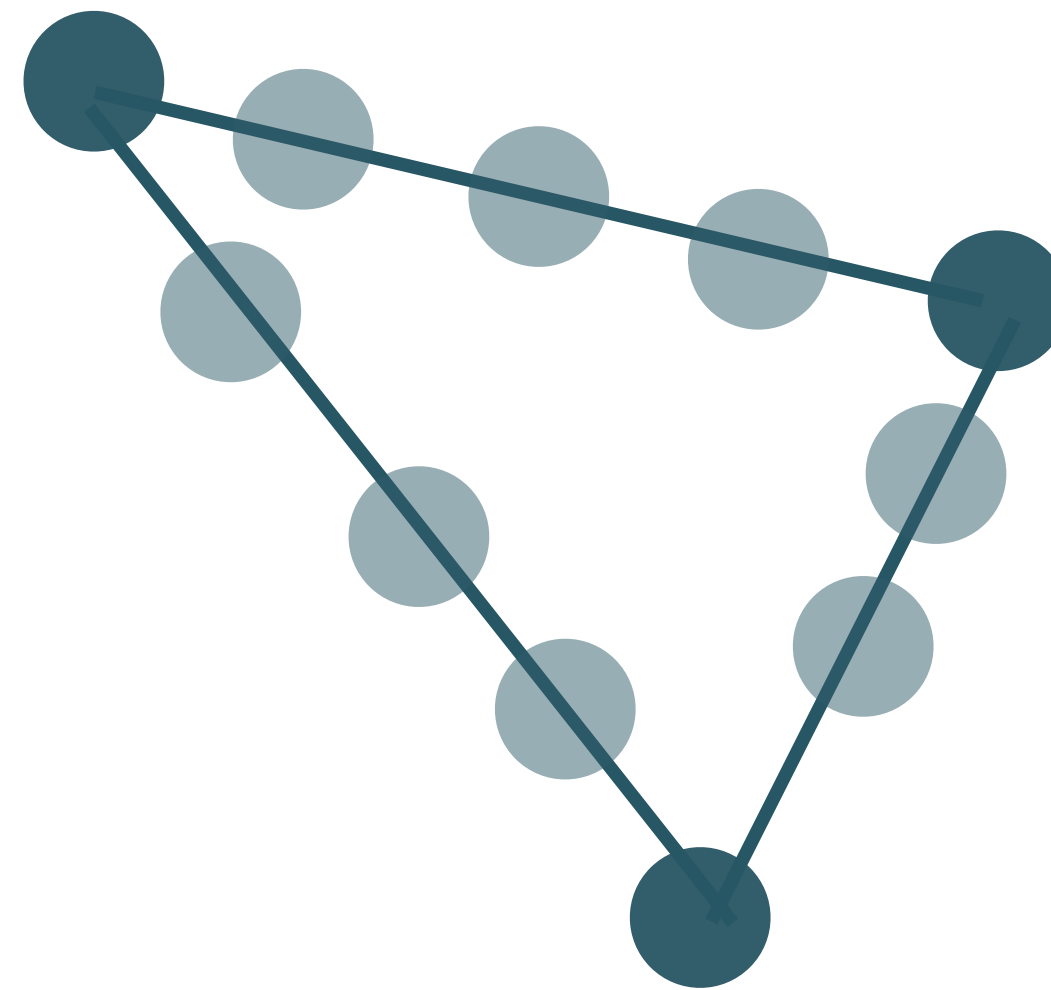
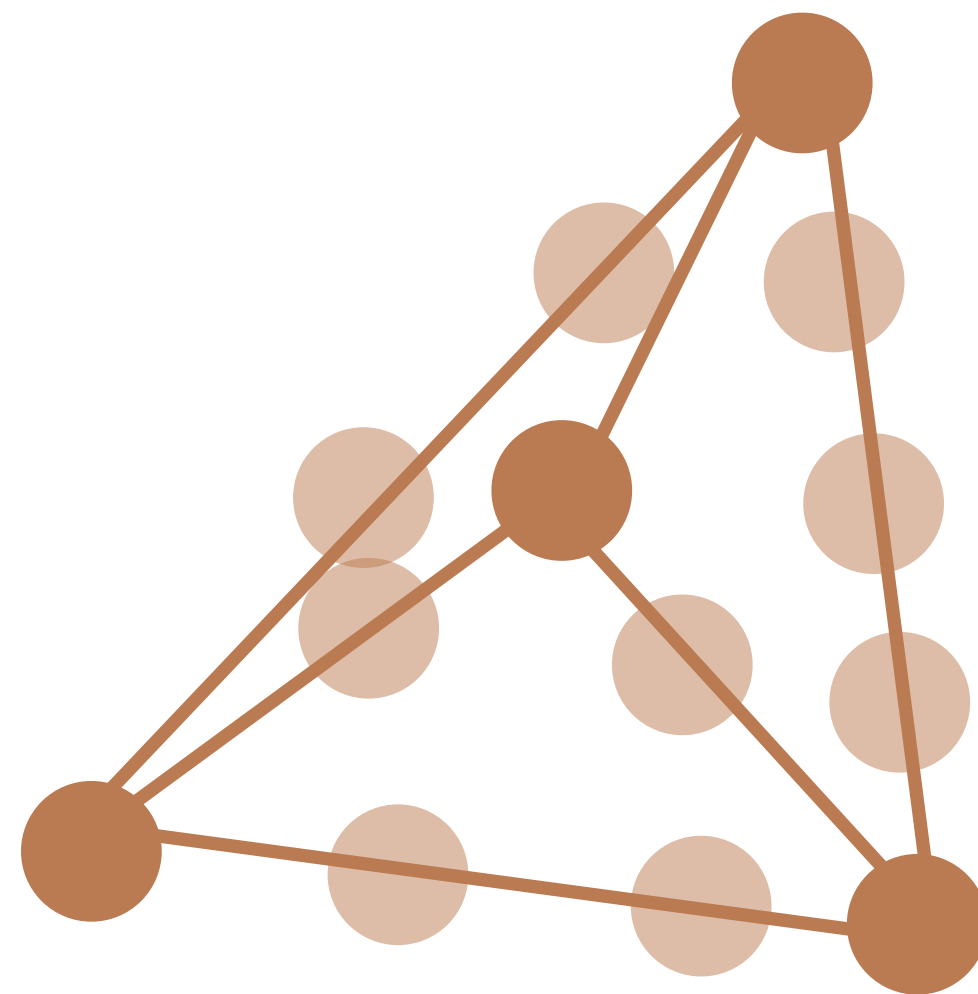
Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

SMOTE: Synthetic Minority Over-sampling Technique



Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

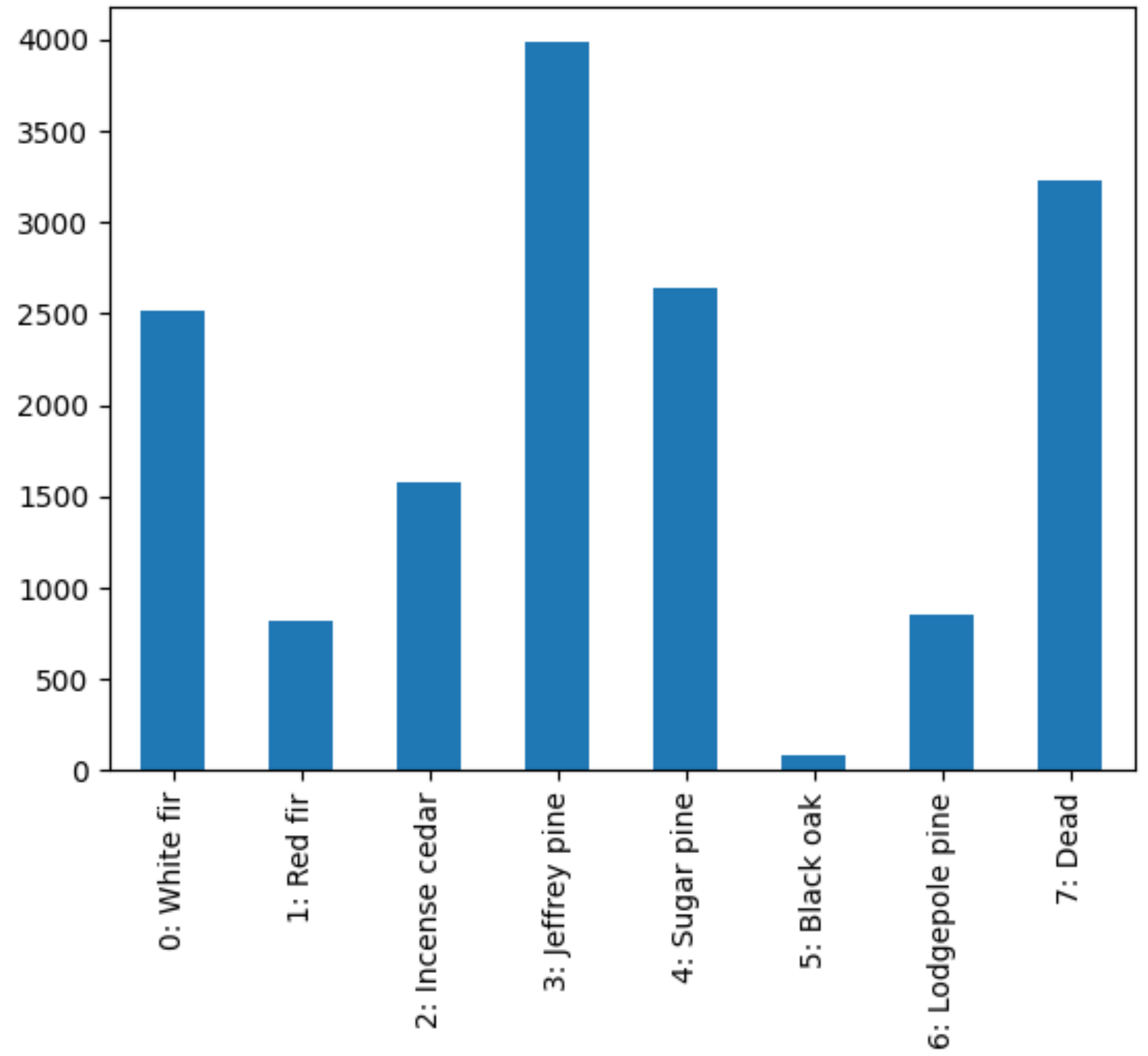
SMOTE: Synthetic Minority Over-sampling Technique



Nitesh V Chawla, Kevin W Bowyer, Lawrence O Hall, and W Philip Kegelmeyer. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

Class imbalance problem

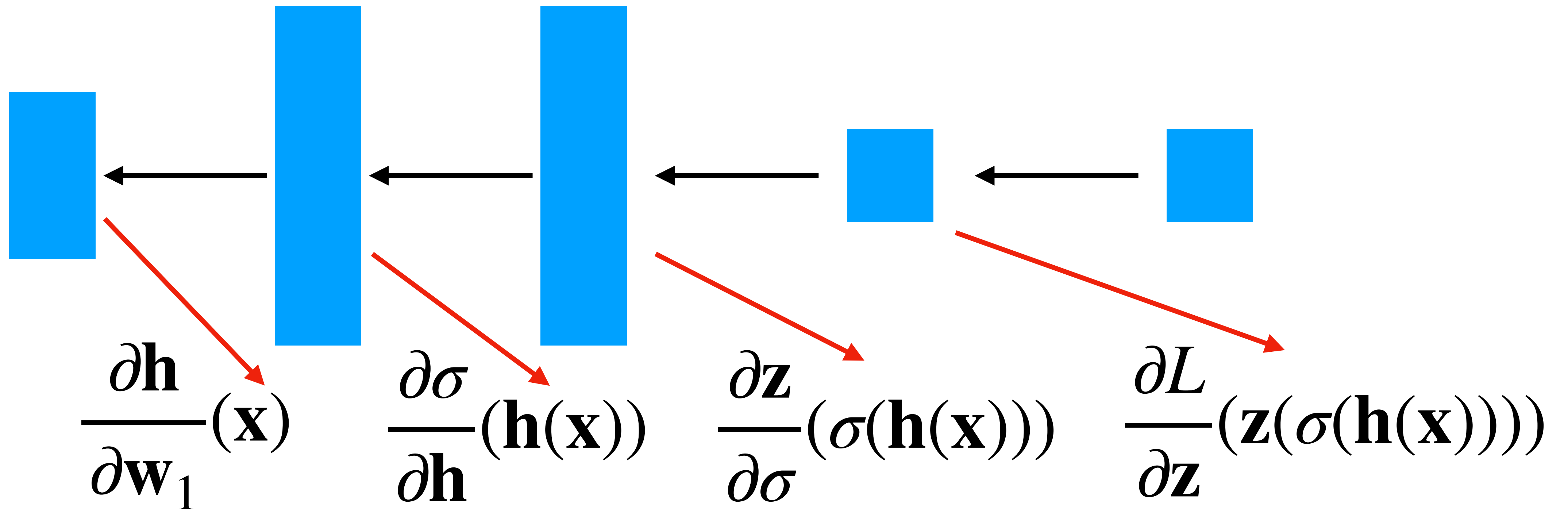
- Datasets often have an unequal number of examples of each class
- How will this affect NN training?
- How could we address the issue?



Dealing with class imbalance

- Oversampling and undersampling
- Class weights
- Data augmentation

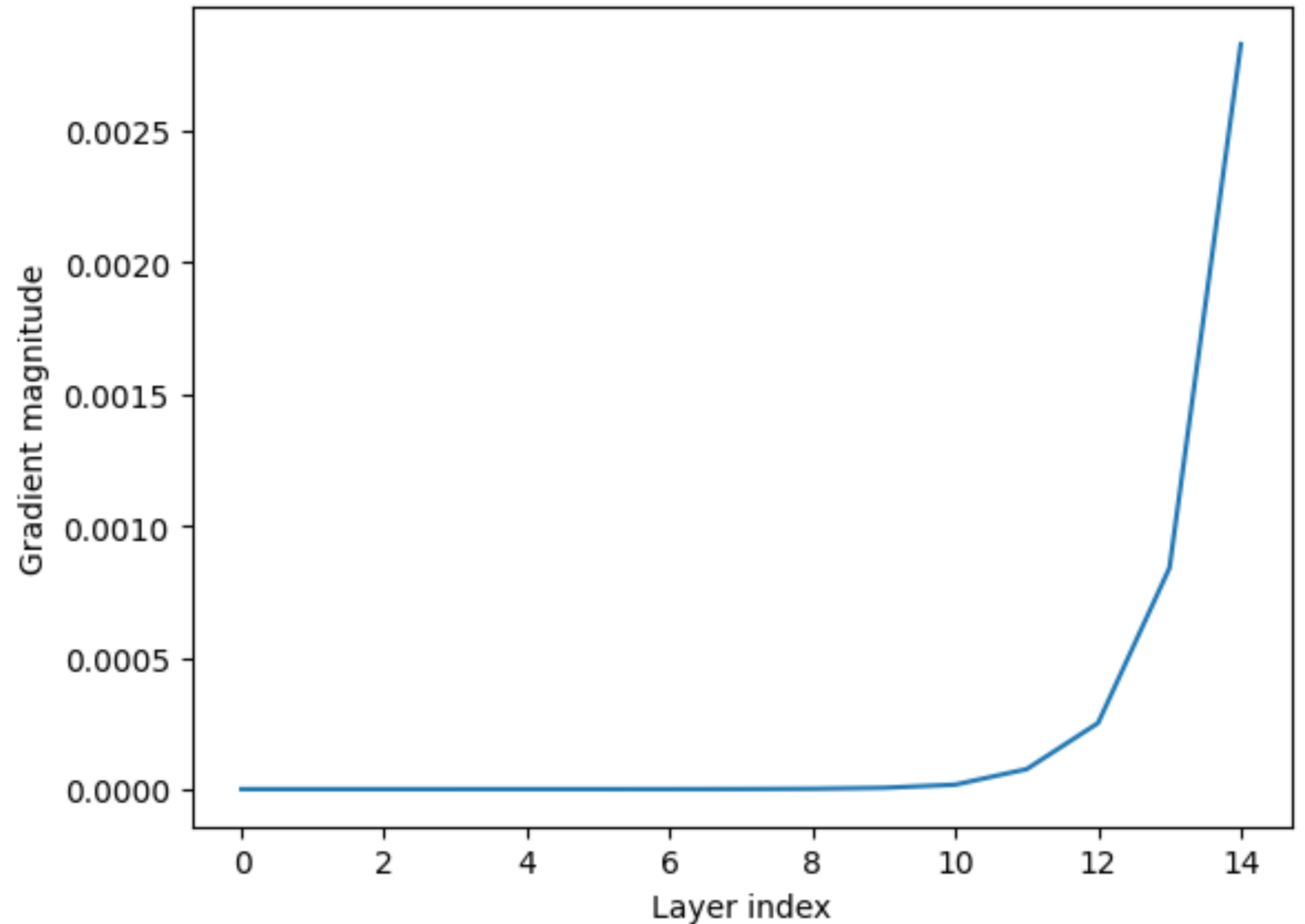
“Vanishing gradient” problem



Early layers have smaller gradients than later ones.

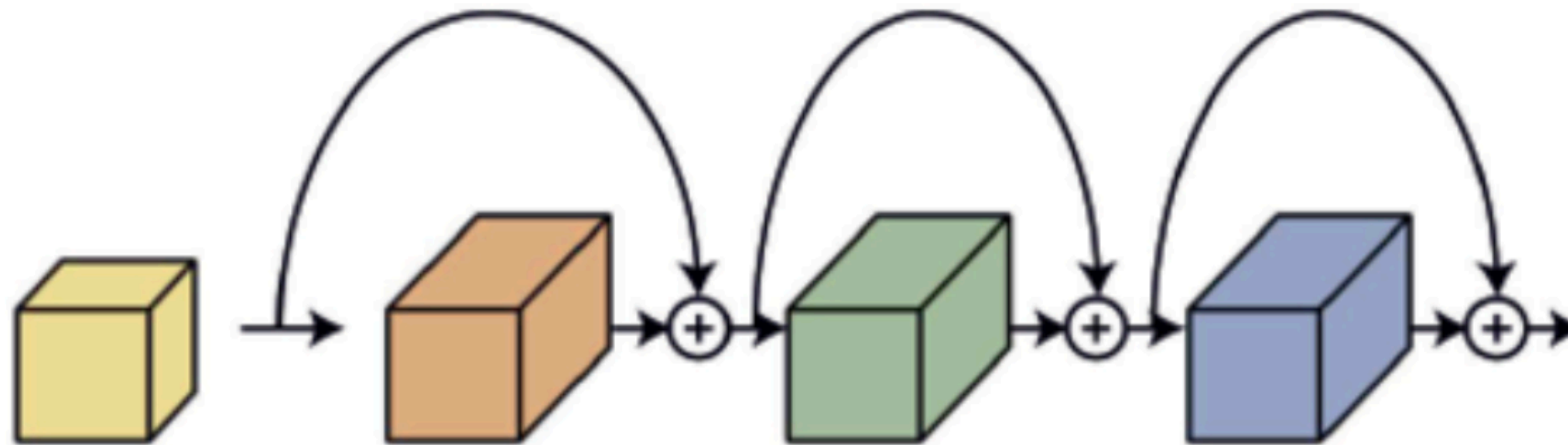
“Vanishing gradient” problem: example

- 15-layer network



Residual connections

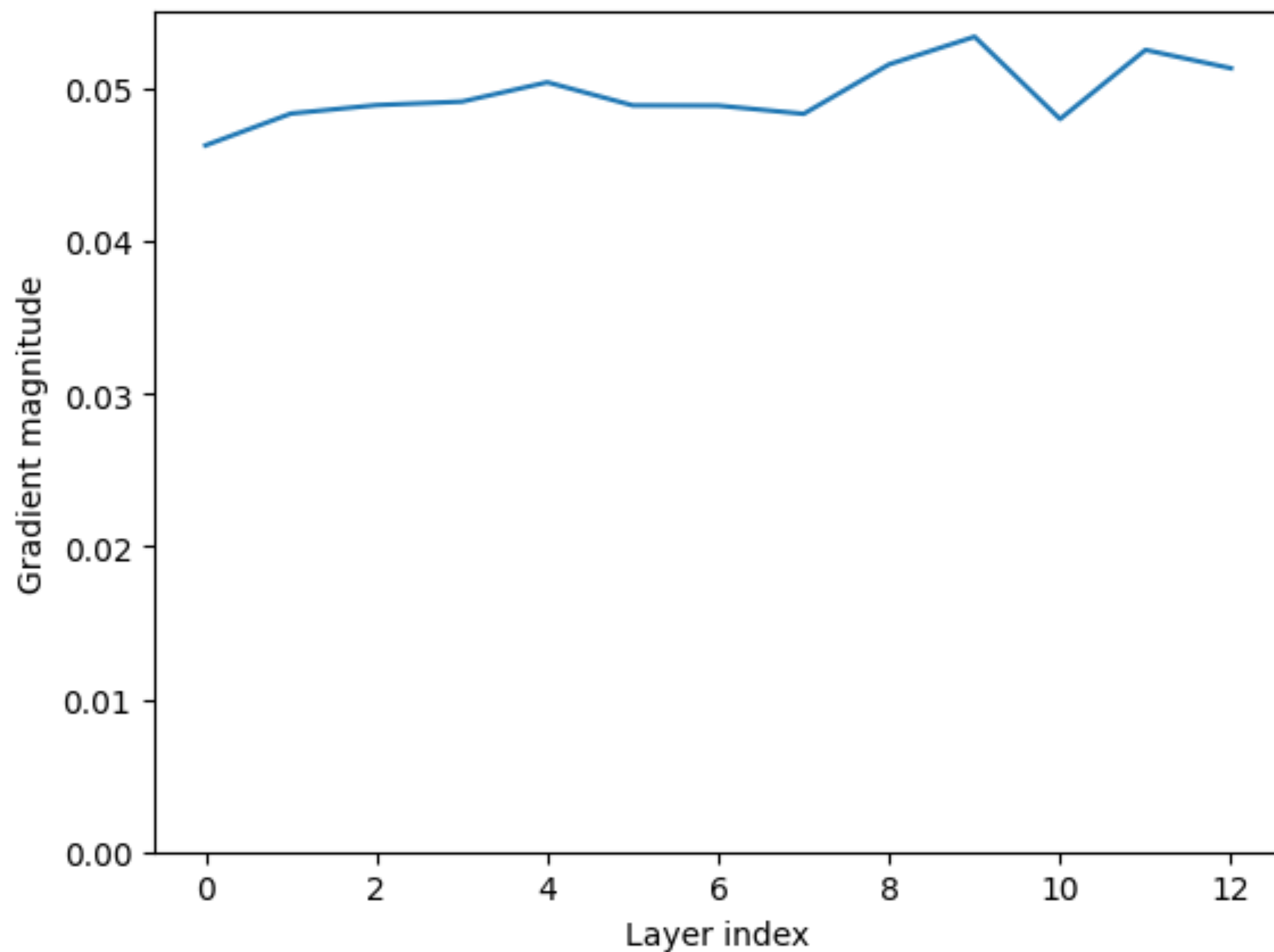
- A solution for the “vanishing gradient” problem
- Idea: add “skip connections” between layers



$$\mathbf{h}_{i+1}(\mathbf{h}_i) = f_i(\mathbf{h}_i) + \mathbf{h}_i$$

Residual connections: example

- 15-layer network



Residual connections: case study on CA housing dataset

# Hidden Layers	Residual Connections?	Train MSE	Test MSE
3	No	0.174	0.226
3	Yes	0.190	0.233
15	No	0.456	0.492
15	Yes	0.134	0.223

Implementing residual connections

```
class LinearWithResidual(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        self.linear = nn.Linear(in_features, out_features)
        self.act = nn.SiLU()

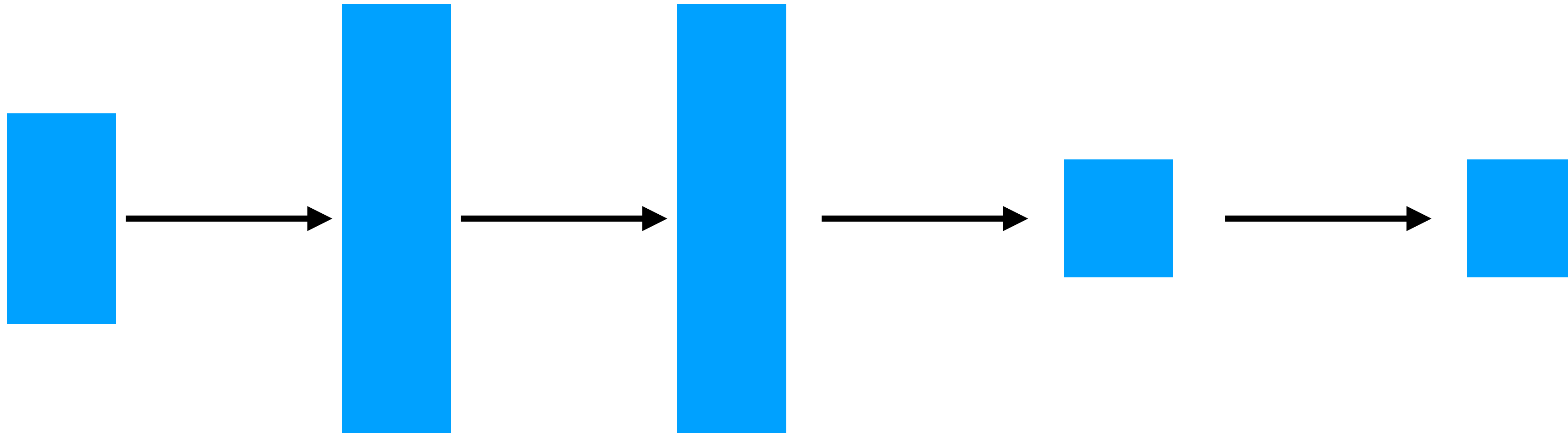
    def forward(self, x):
        z = self.act(self.linear(x)) + x
        return z

nn_model = nn.Sequential(
    nn.Linear(in_size, hidden_size),
    LinearWithResidual(hidden_size, hidden_size),
    nn.Linear(hidden_size, out_size)
)
```

Data Normalization

- Typically we want to normalize our data in some way:
 - Zero mean
 - Unit std. dev.
- Why is this important?

“Internal Covariate Shift” problem



Distribution of each layer's inputs changes during training

Batch normalization

- Idea: normalize layer output using per-batch mean and variance
- Batch normalization with input x :

$$z = \frac{x - \text{mean}(x)}{\sqrt{\text{var}(x) + \epsilon}} \cdot \gamma + \beta$$

- Also maintains a running mean and variance to be used at inference time.

Batch normalization: case study on California housing dataset

# Hidden Layers	Data normalization?	Batch normalization?	Train MSE	Test MSE
3	Yes	No	0.310	0.346
3	No	No	1.312	1.319
3	No	Yes	0.365	0.389
3	Yes	Yes	0.155	0.253